Reliability Engineering Project

SLOs, KPIs, Dashboards, Proactive Alerting and 24x7 Incident Response

The project statement

Cut MTTR 45% and paged incidents 38%, as measured by postmortems, by establishing SLOs, KPIs, dashboards, and proactive alerting, leading 24x7 incident diagnosis and root-cause analysis, and mentoring 6 engineers in operational excellence.

A plain-language explanation of every component, the end-to-end workflow and the architecture

Contents

1. Overall Summary in Simple Terms
2. Key Terms Made Simple
3. Architecture Diagram (with layer-by-layer walkthrough)
4. Every Component Explained in Detail (22 components)
5. End-to-End Workflow (setup + life of one incident)
6. How Each Part Produced the Results
7. Final Recap

1. Overall Summary in Simple Terms

Imagine a large online shop that must stay open day and night. Sometimes things break: pages load slowly, payments fail, or the whole site goes down. Each time that happens, an engineer's phone rings (a "page"), they wake up, find the problem, and fix it.

This project made that whole process faster, calmer and less frequent. In plain words, it did four things:

1. **Decided what "good" looks like.** We wrote clear promises called SLOs, for example "99.9% of checkouts succeed" and "95% of pages load in under 300 milliseconds". We also chose KPIs, the scorecard numbers that tell us how well the team handles problems (such as how long a fix takes).
2. **Made the system visible.** We collected health signals (numbers, logs and request journeys) from every part of the system and showed them on dashboards, so anyone can see at a glance what is healthy and what is not.
3. **Warned early, and only when it matters.** We replaced noisy alarms that fired for every small blip with smart, proactive alerts that fire when a promise is truly at risk, often before customers notice.
4. **Got better at fixing and learning.** We led round-the-clock (24x7) incident response, found the real root causes, wrote blame-free postmortems, and coached 6 engineers so the whole team could do this well.

1.1 The Results

Result	What it means	Simple example
MTTR cut by 45%	Mean Time To Recovery: the average time from "something broke" to "it is working again" became 45% shorter.	An outage that used to take 60 minutes to fix now takes about 33 minutes.
Paged incidents cut by 38%	38% fewer times an engineer was woken up or interrupted by an urgent alert.	If engineers were paged 100 times a month before, it is now about 62 times.
Measured by postmortems	The numbers come from written incident reports with exact timelines, not from guesses.	Each postmortem records when the issue started, was detected and was fixed.

Why the two numbers improved

MTTR dropped because problems were spotted earlier (better alerts), understood faster (dashboards, traces and runbooks) and handled by a well-trained team (mentoring and clear incident roles).

Pages dropped because noisy, useless alerts were removed and because the root causes of repeat problems were permanently fixed after each postmortem.

2. Key Terms Made Simple

These words appear throughout the document. Each is explained with an everyday comparison.

Term	Simple meaning	Everyday comparison
SLI (Service Level Indicator)	A measured number about service health, such as the percentage of successful requests.	Your car's speedometer reading.
SLO (Service Level Objective)	The target for an SLI, such as "99.9% of requests succeed over 30 days".	"I will drive under the speed limit 99.9% of the time."
SLA (Service Level Agreement)	A contract with customers, usually looser than the SLO, with penalties if broken.	A delivery company refunding you if a parcel is late.
Error budget	The amount of failure the SLO allows. 99.9% over 30 days allows about 43 minutes of bad time.	A monthly allowance you can spend; once it is gone, you stop taking risks.
Burn rate	How fast the error budget is being used up compared with normal.	Spending your monthly allowance in two days instead of thirty.
KPI (Key Performance Indicator)	A scorecard number for the team or process, such as MTTR or pages per week.	A school report card.
MTTR	Mean Time To Recovery: average time from start of an incident to full recovery.	How long the plumber takes from leak to fixed pipe.
MTTD / MTTA	Mean Time To Detect / Acknowledge: how long until we notice, and until a person responds.	How long until you notice the leak, and until the plumber answers your call.
Page	An urgent alert that calls the on-call engineer's phone, any time of day.	A fire alarm that wakes you at night.
Incident	Any unplanned event that hurts users or puts an SLO at risk.	The leak itself.
Telemetry	Signals a system sends about itself: metrics, logs and traces.	A car's dashboard lights and trip computer.
Metrics / Logs / Traces	Numbers over time / written event records / the full journey of one request.	Heart-rate chart / a diary / a parcel's tracking history.
Root cause analysis (RCA)	Finding the deepest reason something failed, not just the symptom.	Fixing the cracked pipe, not just mopping the floor.

Term	Simple meaning	Everyday comparison
Postmortem	A blame-free written report after an incident: what happened, why, and how to prevent it.	An accident report focused on lessons, not on punishment.
Runbook	Step-by-step instructions for handling a known problem.	A first-aid guide.
On-call	The rota of engineers responsible for answering pages at any hour.	The doctor on night duty.

3. Architecture Diagram

The diagram below shows every part of the system in six layers, from the services customers use at the top down to learning and improvement at the bottom. Each box has a number that matches its detailed explanation in Section 4.

The diagram is shown on the next page in landscape so every label is readable.

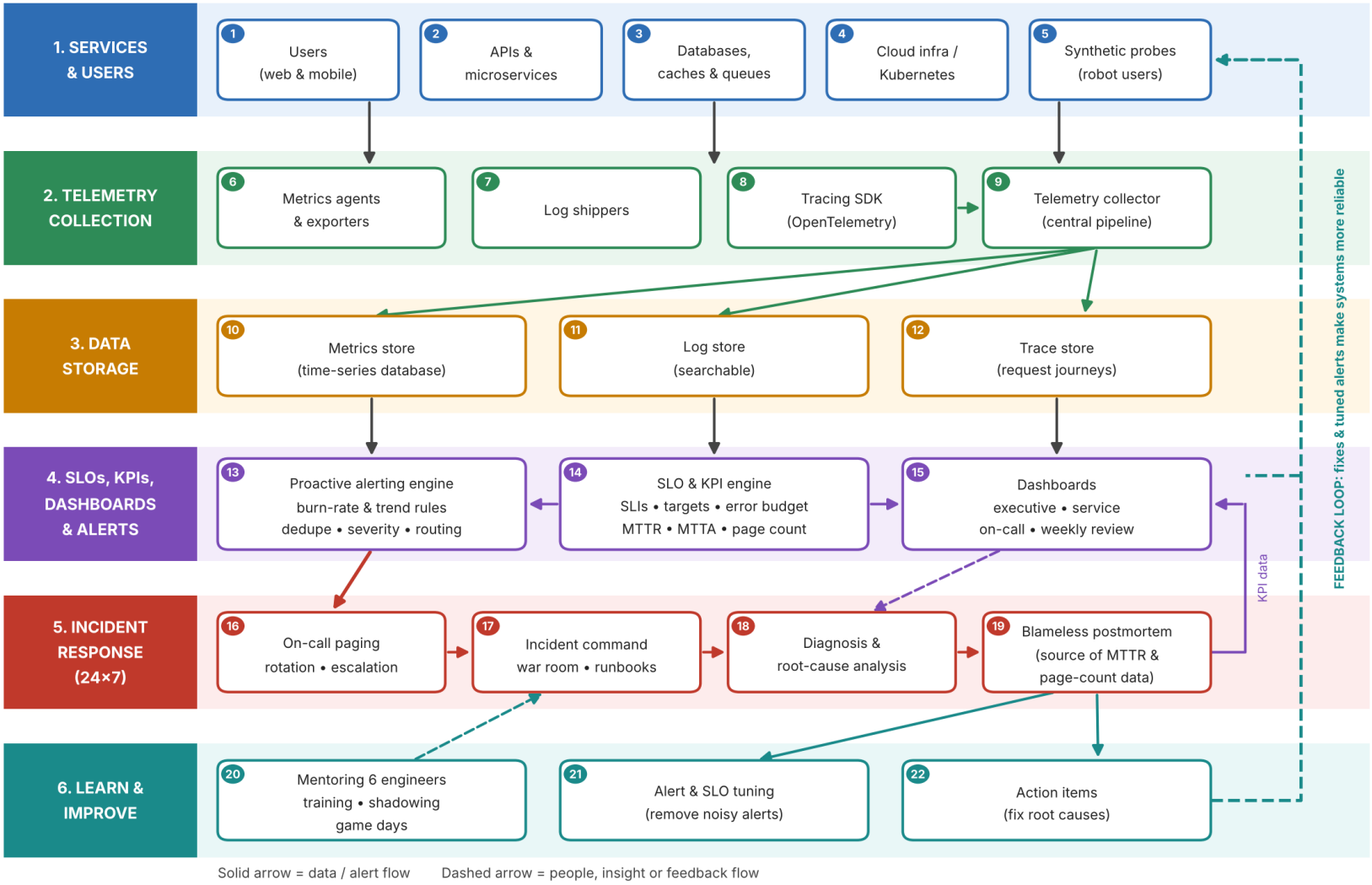


Figure 1. End-to-end architecture of the reliability system

3.1 How to Read the Diagram

- **Coloured bands (rows):** each band is one layer. Data generally flows from top to bottom.
- **Numbered boxes:** each box is one component; the number links it to Section 4.
- **Solid arrows:** machine data or alerts moving automatically from one part to another.
- **Dashed arrows:** people, insight or improvements moving back into the system.
- **Right-hand dashed loop:** the feedback loop. Lessons from incidents flow back up to make services and alerts better.
- **Purple "KPI data" line:** numbers from postmortems (MTTR, page counts) feeding the dashboards.

3.2 Layer-by-Layer Walkthrough of the Diagram

Layer 1: Services and Users (boxes 1 to 5)

This is the real product. Users (1) click buttons in web and mobile apps. Their requests reach APIs and microservices (2), which read and write data in databases, caches and queues (3). All of this runs on cloud infrastructure and Kubernetes (4). Synthetic probes (5) are small robot programs that act like users every minute, so we learn about problems even at 3 a.m. when real traffic is low. The arrows leaving this layer show that every one of these parts produces health signals.

Layer 2: Telemetry Collection (boxes 6 to 9)

Small helper programs pick up the health signals. Metrics agents and exporters (6) gather numbers such as request counts and CPU use. Log shippers (7) gather written event records. The tracing SDK (8) tags each request with an ID so its full journey can be followed. Everything goes into the telemetry collector (9), a central pipeline that cleans the data, adds labels (service name, region, version) and sends it to storage. The arrow from 8 to 9 shows signals joining the central pipe.

Layer 3: Data Storage (boxes 10 to 12)

Three specialised stores keep the signals. The metrics store (10) is a time-series database, built for numbers over time. The log store (11) is a searchable archive of events. The trace store (12) keeps request journeys. The three green arrows from the collector show the data being sorted into the right store, like mail into the right pigeonholes.

Layer 4: SLOs, KPIs, Dashboards and Alerts (boxes 13 to 15)

This is the brain. The SLO and KPI engine (14) in the middle turns raw numbers into meaning: the SLI, the target, how much error budget is left and the team scorecard. It feeds two neighbours (purple arrows): the proactive alerting engine (13), which decides whether a person must be woken up, and the dashboards (15), which display everything to people. The stores feed this layer from above.

Layer 5: Incident Response 24x7 (boxes 16 to 19)

When the alerting engine decides something is serious, the thick red arrow carries a page to the on-call system (16). The engineer joins incident command (17) and follows a runbook. They move to diagnosis and root-cause analysis (18), using the dashboards (the dashed purple arrow from 15). After recovery, the team writes a blameless postmortem (19). The postmortem is also where the official MTTR and page counts are recorded, which is why a purple line carries "KPI data" back up to the dashboards.

Layer 6: Learn and Improve (boxes 20 to 22)

The postmortem creates two kinds of follow-up (teal arrows). Alert and SLO tuning (21) deletes or adjusts noisy alerts, which reduces pages. Action items (22) fix the real root causes, which stops repeat incidents. Mentoring the 6 engineers (20) feeds better skills into incident command (dashed arrow to 17). The long dashed feedback loop on the right carries all these improvements back to the services (Layer 1) and to the alert rules (Layer 4), so the whole system gets more reliable each cycle.

4. Every Component Explained in Detail

Each component below follows the same pattern: what it is in one line, an everyday comparison, how it works step by step, the small details that matter, and why it helped achieve the results.

4.1 Layer 1: Services and Users

1. Users (web and mobile)

In one line: The real customers whose experience the whole project protects.

Everyday analogy: *Shoppers walking into a store.*

How it works, step by step:

- A user opens the app and taps a button, for example "Pay now".
- The app sends a request over the internet to our servers.
- The user waits; if the answer is slow or an error, they have a bad experience.
- Front-end code records small timings (page load time, button response time) and sends them back as real-user monitoring data.

Minute details:

- Only user-visible symptoms (slow, broken, wrong) are used for SLOs, because those are what customers feel.
- Critical user journeys were listed first: login, search, add to cart, checkout, payment.

Why it matters for the results: Starting from the user meant alerts fire for real customer pain rather than internal noise, which cut useless pages.

2. APIs and Microservices

In one line: The many small programs that do the actual work, each handling one job.

Everyday analogy: *Different counters in a store: one for payments, one for returns, one for stock checks.*

How it works, step by step:

- A request arrives at an API gateway, the front door.
- The gateway passes it to the right microservice, for example the payment service.
- That service may call other services and the database before answering.
- Each service is instrumented: it counts requests, errors and response time, writes logs and passes on a trace ID.

Minute details:

- The "golden signals" were tracked for each service: latency (speed), traffic (load), errors and saturation (how full the service is).

- Health-check endpoints let systems ask "are you alive?" every few seconds.

Why it matters for the results: Consistent signals from every service made it possible to locate a failing service in minutes, which shortened MTTR.

3. Databases, Caches and Queues

In one line: Where data is stored (databases), kept close for speed (caches) and lined up for later processing (queues).

Everyday analogy: *The warehouse (database), the shelf by the till (cache) and the queue of orders waiting to be packed (queue).*

How it works, step by step:

- Services read and write records in the database.
- Frequently used data is copied to a cache so it can be served quickly.
- Slow jobs, such as sending emails, are placed in a queue and handled in the background.
- Exporters measure query time, connection counts, cache hit rate and queue length.

Minute details:

- A rising queue length is an early warning sign that work is piling up.
- A falling cache hit rate often explains sudden database slowness.

Why it matters for the results: Many incidents start here. Watching these leading indicators enabled proactive alerts before users were affected.

4. Cloud Infrastructure and Kubernetes

In one line: The servers, networks and container platform that everything runs on.

Everyday analogy: *The building, electricity and plumbing of the store.*

How it works, step by step:

- Kubernetes runs services in containers and restarts them if they crash.
- The cloud provides virtual machines, load balancers and storage.
- Infrastructure metrics (CPU, memory, disk, network, pod restarts) are collected continuously.
- Deployment events (new versions released) are recorded so they can be matched with problems.

Minute details:

- Recording every deployment was important: a large share of incidents begin right after a change.
- Auto-scaling adds capacity when load rises, reducing incidents from traffic spikes.

Why it matters for the results: Linking incidents to recent changes made "roll back the release" a fast first fix, a major contributor to the 45% MTTR cut.

5. Synthetic Probes (Robot Users)

In one line: Small scripts that pretend to be a user and test key journeys every minute from several locations.

Everyday analogy: *A mystery shopper visiting the store every minute, day and night.*

How it works, step by step:

- A probe logs in, searches, and tries a test checkout on a schedule.
- It records whether each step passed and how long it took.
- Results are sent into the telemetry pipeline like any other signal.
- If probes fail from several locations at once, an alert is raised.

Minute details:

- Requiring failures from multiple locations avoids false alarms caused by one bad network path.
- Probes catch problems at night when there is too little real traffic to notice.

Why it matters for the results: Probes shrink detection time (MTTD), which is the first part of MTTR.

4.2 Layer 2: Telemetry Collection

6. Metrics Agents and Exporters

In one line: Programs that collect numbers from each service and machine every few seconds.

Everyday analogy: *A fitness tracker recording your heart rate all day.*

How it works, step by step:

- Each service exposes counters (requests, errors) and timers (response time).
- The agent scrapes (reads) these values on a fixed interval, for example every 15 seconds.
- Each value is stamped with a time and labels such as service, region and version.
- The values are forwarded to the telemetry collector.

Minute details:

- Response times are stored as histograms so percentiles (p95, p99) can be calculated accurately.
- Labels with too many unique values (such as user IDs) were avoided because they overload the database.

Why it matters for the results: Reliable, consistent metrics are the raw material for every SLO and alert.

7. Log Shippers

In one line: Programs that collect the written event records from every service.

Everyday analogy: *A diary of everything that happened, collected from every room.*

How it works, step by step:

- A service writes a log line, for example "payment failed: card declined".
- The shipper reads new lines from each machine or container.
- Logs are converted to a structured format (fields such as time, level, message, trace ID).
- They are sent to the collector and then the log store.

Minute details:

- Structured logs (key-value fields) can be filtered instantly instead of read line by line.
- Including the trace ID in every log line connects logs to traces.

Why it matters for the results: Searchable, linked logs explain the "why" behind a spike in errors, speeding up diagnosis.

8. Tracing SDK (OpenTelemetry)

In one line: A library inside each service that follows one request across every service it touches.

Everyday analogy: *A parcel tracking number showing every depot the parcel passed through.*

How it works, step by step:

- When a request enters, it receives a unique trace ID.
- Each service it passes through records a "span": what it did and how long it took.
- The trace ID is passed along with the request to the next service.
- All spans are sent to the collector and stitched together into one picture.

Minute details:

- Sampling keeps every error trace and a portion of normal traces, controlling cost.
- OpenTelemetry is an open standard, so the same code works with many tools.

Why it matters for the results: Traces show exactly which hop was slow or failing, often turning an hour of guessing into minutes.

9. Telemetry Collector (Central Pipeline)

In one line: One central pipeline that receives all metrics, logs and traces, tidies them and routes them to storage.

Everyday analogy: *A sorting office for mail.*

How it works, step by step:

- It receives signals from all agents, shippers and SDKs.
- It cleans them: removes duplicates, drops junk and masks sensitive data.
- It enriches them with labels: team owner, environment, region, version.
- It sends each signal type to its own store.

Minute details:

- Buffering and retries prevent data loss if a store is briefly unavailable.
- The collector is monitored itself, so a blind spot is noticed immediately.

Why it matters for the results: Consistent labels like "owning team" let alerts reach the right person first time, cutting acknowledge time.

4.3 Layer 3: Data Storage

10. Metrics Store (Time-Series Database)

In one line: A database built to store and query numbers over time very quickly.

Everyday analogy: *A giant spreadsheet with one column per minute.*

How it works, step by step:

- Metric values arrive with time stamps and labels.
- They are compressed and stored efficiently.
- Queries such as "error rate of checkout over the last hour" return in seconds.
- Old detailed data is rolled up into summaries to save space.

Why it matters for the results: Fast queries power both live dashboards and alert rules that run every minute.

11. Log Store (Searchable)

In one line: A searchable archive of all log events.

Everyday analogy: *A library catalogue for the diary entries.*

How it works, step by step:

- Logs are indexed by fields such as service, level and trace ID.
- Engineers search, for example "errors in payment service in the last 15 minutes".
- Results can be grouped to show the most common error messages.
- Retention rules decide how long logs are kept.

Why it matters for the results: Quick searches find the exact error message behind an incident.

12. Trace Store (Request Journeys)

In one line: A store of complete request journeys, searchable by trace ID or by slowness.

Everyday analogy: *A record of every parcel's route.*

How it works, step by step:

- Spans are assembled into full traces.
- Engineers open a trace to see a timeline of every step.
- Slow or failed steps are highlighted.

- From a trace, one click jumps to the matching logs.

Why it matters for the results: Pinpoints the failing component, cutting the investigation part of MTTR.

4.4 Layer 4: SLOs, KPIs, Dashboards and Alerts

14. SLO and KPI Engine (explained first because the others depend on it)

In one line: The part that turns raw numbers into clear answers: "Are we keeping our promises?" and "How well is the team responding?"

Everyday analogy: *A referee keeping score against the rules of the game.*

How it works, step by step:

- Choose SLIs for each critical user journey, for example "percentage of checkout requests that succeed in under 300 ms".
- Set an SLO target with a time window, for example 99.9% over a rolling 30 days.
- Calculate the error budget: 0.1% of 30 days is about 43 minutes of allowed bad time.
- Continuously compute how much budget is used and the current burn rate.
- Compute team KPIs from incident and postmortem data: MTTR, MTTD, MTTA, pages per week, repeat incidents and action-item completion.

Minute details:

- SLOs were agreed with product owners so they reflect what customers truly need, not perfection.
- An error-budget policy was set: if the budget runs out, the team pauses risky releases and focuses on reliability.
- SLO definitions were kept as code, reviewed like any other change.

Why it matters for the results: With clear targets, the team knows what is worth waking up for and what can wait until morning; this single decision drove much of the 38% page reduction.

Example SLI	Example SLO	Error budget (30 days)
Checkout success rate	99.9% of requests succeed	About 43 minutes of failure
Search latency	95% of searches under 300 ms	5% of searches may be slower
Login availability	99.95% of logins succeed	About 22 minutes of failure

13. Proactive Alerting Engine

In one line: The rules that decide when to wake a person up, when to send a quiet ticket, and when to stay silent.

Everyday analogy: *A smoke detector that ignores burnt toast but screams for a real fire.*

How it works, step by step:

- Every minute, alert rules query the metrics store.
- Burn-rate rules check how fast the error budget is being spent, using a short and a long window together.
- Fast burn (for example 14 times normal over 1 hour, confirmed over 5 minutes) sends an urgent page.
- Slow burn (for example 3 to 6 times normal over 6 hours) creates a ticket for working hours.
- Trend rules watch leading indicators (disk filling, queue growing, certificate expiring) and warn days or hours ahead.
- Duplicate alerts are grouped into one; each alert gets a severity and is routed to the owning team with a runbook link.

Minute details:

- Old alerts on raw causes ("CPU above 80%") were reviewed; those that never needed human action were deleted or turned into tickets.
- Every page must be urgent, actionable and user-affecting; otherwise it is not a page.
- Maintenance windows silence alerts during planned work.

Why it matters for the results: Fewer, smarter alerts directly reduced paged incidents by 38%, and early warnings let teams fix issues before they became outages.

15. Dashboards

In one line: Visual screens that show health and scorecards to different audiences.

Everyday analogy: *The instrument panel of a plane, with a simple view for passengers and a detailed one for pilots.*

How it works, step by step:

- Executive dashboard: SLO status for each product (green, amber, red) and monthly KPIs.
- Service dashboards: golden signals, dependencies and recent deployments for each service.
- On-call dashboard: active alerts, error budget burn and quick links to logs and traces.
- Weekly review dashboard: MTTR trend, pages per week, top noisy alerts and open action items.

Minute details:

- Every dashboard followed the same layout so engineers know where to look under pressure.
- Deployment markers on graphs show at a glance whether a release caused a change.
- Links from each graph jump straight to related logs and traces.

Why it matters for the results: During an incident, a good dashboard answers "what is broken and since when" in seconds, reducing diagnosis time.

4.5 Layer 5: Incident Response 24x7

16. On-Call Paging (Rotation and Escalation)

In one line: The system and schedule that make sure a trained person always answers an urgent alert.

Everyday analogy: *The doctor-on-duty rota at a hospital.*

How it works, step by step:

- A rota lists the primary and secondary on-call engineer for each week.
- A page goes to the primary's phone by push notification, SMS and call.
- If not acknowledged within a set time (for example 5 minutes), it escalates to the secondary, then to the manager.
- The acknowledge time is recorded automatically.

Minute details:

- Clear hand-over notes at each shift change.
- Pages per shift were tracked to protect engineers from burnout.

Why it matters for the results: Guaranteed fast acknowledgement removes waiting time from MTTR.

17. Incident Command (War Room and Runbooks)

In one line: A clear, calm way to run an incident with defined roles and steps.

Everyday analogy: *A fire brigade with a commander, crew and radio operator.*

How it works, step by step:

- The responder declares an incident and sets a severity (SEV1 to SEV4).
- A dedicated chat channel and video call (the war room) open automatically.
- Roles are assigned: Incident Commander (decides), Operations Lead (fixes), Communications Lead (updates stakeholders), Scribe (keeps the timeline).
- The team follows the runbook linked in the alert.
- Mitigation comes first: stop the user pain (roll back, fail over, add capacity), then investigate deeply.

Minute details:

- Status updates go to stakeholders at a fixed rhythm, for example every 30 minutes.
- The timeline is written live, which later feeds the postmortem.

Why it matters for the results: Clear roles prevent confusion and duplicated work; "mitigate first" shortens recovery even before the root cause is known.

18. Diagnosis and Root-Cause Analysis

In one line: Finding what is broken now, then the deeper reason it broke.

Everyday analogy: *A doctor treating symptoms first, then running tests to find the illness.*

How it works, step by step:

- Check the SLO dashboard: which user journey is affected and since when.
- Check recent changes: deployments, configuration changes, traffic spikes.
- Follow traces to find the failing service.
- Read logs from that service to find the exact error.
- After recovery, use the "5 Whys" and a timeline to reach the root cause and contributing factors.

Minute details:

- Example 5 Whys: Why did checkout fail? Database timeouts. Why? Connections ran out. Why? A new release opened connections without closing them. Why? No test covered it. Why? No load test step in the release pipeline. Root cause: missing load test.
- This work was led around the clock (24x7), with the lead joining or guiding major incidents at any hour.

Why it matters for the results: Faster diagnosis lowers MTTR now; true root-cause fixes stop the same incident from paging again.

19. Blameless Postmortem

In one line: A written review after every significant incident, focused on learning, not blame.

Everyday analogy: *A flight-safety report after a near miss.*

How it works, step by step:

- Write a summary: what happened and who was affected.
- Record an exact timeline: start, detected, acknowledged, mitigated, resolved.
- Explain root cause and contributing factors.
- List what went well and what went badly.
- Create action items with an owner and a due date.
- Review it in a team meeting and share it widely.

Minute details:

- The timeline timestamps are where the official MTTR, MTTD and MTTA are measured; this is what "as measured by postmortems" means.
- Blameless language ("the system allowed" rather than "Bob broke") encourages honesty.

Why it matters for the results: Accurate measurement proved the 45% and 38% improvements, and the action items are what prevent repeat pages.

4.6 Layer 6: Learn and Improve

20. Mentoring 6 Engineers in Operational Excellence

In one line: Coaching six engineers so reliability skills spread across the whole team.

Everyday analogy: *A senior pilot training junior pilots in a simulator and on real flights.*

How it works, step by step:

- Training sessions on SLOs, alert design, dashboards and incident roles.
- Shadowing: a mentee joins on-call alongside an experienced engineer, then leads with the mentor watching (reverse shadowing).
- Game days: practice incidents are created on purpose in a safe environment to rehearse response.
- Mentees write and present postmortems and runbooks with feedback.
- Regular one-to-one sessions to track progress.

Minute details:

- Progress was tracked with a skills checklist: can lead an incident, can write an SLO, can tune an alert.
- More trained engineers means less reliance on one expert at 3 a.m.

Why it matters for the results: A larger pool of capable responders made every incident faster to resolve and made the improvements last.

21. Alert and SLO Tuning

In one line: Regularly reviewing alerts and targets to keep them useful.

Everyday analogy: *Adjusting a smoke detector that keeps going off when you cook.*

How it works, step by step:

- Weekly review of every page from the past week.
- For each page ask: was it real, urgent and actionable?
- Delete, merge, downgrade to a ticket, or adjust thresholds for noisy alerts.
- Review SLO targets quarterly against user needs.

Why it matters for the results: This continuous clean-up is a direct cause of the 38% reduction in paged incidents.

22. Action Items (Fix Root Causes)

In one line: The concrete engineering tasks that come out of postmortems.

Everyday analogy: *Replacing the faulty wiring after the fuse blows, not just resetting the fuse.*

How it works, step by step:

- Each action item has an owner, due date and priority.

- Items are tracked in the team's normal work board.
- Completion rate is a KPI on the weekly dashboard.
- Repeat incidents with the same cause are flagged as a warning sign.

Why it matters for the results: Fixed root causes mean the same problem does not page anyone again.

5. End-to-End Workflow

There are two workflows. The first is the one-time setup that built the system. The second runs every time something goes wrong, around the clock.

5.1 Workflow A: Building the Reliability System (Setup)

Step 1: Measure the starting point (baseline)

- Collected past incidents and postmortems.
- Calculated the old MTTR and number of pages per week.
- Listed every existing alert and how often it fired.
- Simple meaning: you cannot show improvement unless you know where you started.

Step 2: Identify critical user journeys

- Worked with product owners to list what customers care about most: login, search, checkout, payment.
- Ranked them by business impact.

Step 3: Define SLIs and SLOs

- For each journey, chose an SLI (success rate, latency).
- Agreed an SLO target and a 30-day window.
- Calculated error budgets and wrote an error-budget policy.

Step 4: Instrument everything

- Added metrics, structured logs and tracing to each service.
- Deployed agents, log shippers and the central collector.
- Set up synthetic probes for each critical journey.

Step 5: Define KPIs

- Chose team scorecard numbers: MTTR, MTTD, MTTA, pages per week, repeat incidents, action-item completion.
- Standardised the postmortem template so every timeline captures the same timestamps.

Step 6: Build dashboards

- Executive, service, on-call and weekly-review dashboards with a consistent layout.
- Added deployment markers and links to logs and traces.

Step 7: Redesign alerting

- Replaced cause-based alerts with SLO burn-rate alerts.
- Added trend-based early warnings.
- Every page got a severity, an owner and a runbook link; non-urgent alerts became tickets.

Step 8: Set up on-call and incident process

- Created rotations with escalation.
- Defined severity levels and incident roles.
- Wrote runbooks for common problems.

Step 9: Train the team

- Mentored 6 engineers through training, shadowing and game days.

Step 10: Review and improve continuously

- Weekly alert reviews, monthly KPI reviews, quarterly SLO reviews.
- Compared KPIs with the baseline to show the 45% MTTR and 38% page reductions.

5.2 Workflow B: The Life of One Incident (Runs 24x7)

Here is one realistic example followed from start to finish, with the time each step happens.

> **Example scenario**
> At 02:00 a new version of the payment service is released. It has a hidden bug that slowly uses up database connections. Customers begin to see checkout failures.

Step 1: Problem starts (02:00)

The bug begins; a small share of checkouts fail. Users and synthetic probes start getting errors.

Step 2: Signals are produced (02:00 to 02:01)

Payment-service metrics show errors rising. Logs show "connection pool exhausted". Traces show slow database calls. The probe from two locations fails.

Step 3: Signals are collected and stored (within seconds)

Agents, shippers and the SDK send data to the collector, which labels it ("service: payment, team: payments, version: 2.4") and stores it.

Step 4: SLO engine sees the risk (02:03)

The checkout SLI drops. The error budget is burning about 14 times faster than normal.

Step 5: Proactive alert fires (02:04)

The fast-burn rule is true for both the 1-hour and 5-minute windows. One grouped page is created with severity SEV2, owner "payments" and a runbook link. Detection time (MTTD) is about 4 minutes.

Step 6: On-call is paged and acknowledges (02:06)

The primary on-call engineer's phone rings. They acknowledge within 2 minutes, so no escalation is needed.

Step 7: Incident is declared (02:08)

A war-room channel opens. Roles are assigned: Incident Commander, Operations Lead, Communications Lead and Scribe. The first status update goes out.

Step 8: Quick diagnosis (02:08 to 02:15)

The on-call dashboard shows errors started at 02:00, matching a deployment marker. Traces point to the payment service's database calls. Logs confirm connection exhaustion.

Step 9: Mitigate first (02:16)

Following the runbook, the team rolls back to version 2.3. Mitigation before deep investigation stops customer pain fast.

Step 10: Verify recovery (02:16 to 02:26)

Error rate returns to normal, probes pass and the burn rate drops. The incident is marked resolved at 02:26. MTTR is 26 minutes.

Step 11: Root-cause analysis (next working day)

Using the 5 Whys, the team finds the code path that did not release connections and that no load test caught it.

Step 12: Blameless postmortem (within 2 to 3 days)

The timeline from the Scribe gives exact timestamps for MTTD, MTTA and MTTR. Action items: fix the connection leak, add a load test to the release pipeline, add an early-warning alert on connection-pool usage.

Step 13: Learn and improve (following weeks)

Action items are completed and tracked. The new early-warning alert means a similar issue would be caught before customers notice. A mentee presents the postmortem as a learning exercise.

Step 14: KPIs updated (weekly and monthly)

The incident's numbers flow to the weekly review dashboard. Over many incidents, these numbers show MTTR falling 45% and pages falling 38%.

5.3 Incident Timeline at a Glance

Time	Event	Metric captured
02:00	Bad release; failures begin	Incident start time
02:04	Burn-rate alert fires	Time to detect: 4 min
02:06	On-call acknowledges	Time to acknowledge: 2 min after alert
02:08	Incident declared, roles assigned	Severity recorded: SEV2
02:16	Rollback (mitigation)	Time to mitigate: 16 min
02:26	Recovery verified, resolved	Time to recover (MTTR input): 26 min
+2 days	Postmortem reviewed	Root cause and action items recorded

6. How Each Part Produced the Results

This table connects every part of the work to the two headline results.

Work done	Effect on MTTR (45% faster)	Effect on paged incidents (38% fewer)
SLOs and error budgets	Focus on what users feel, so effort goes to real problems	Only real SLO risks page; minor blips do not
KPIs	Made recovery time visible, so it could be managed	Made page volume visible, so noise could be targeted
Dashboards	Answer "what and since when" in seconds	Weekly review highlights noisy alerts to remove
Proactive alerting	Earlier detection; runbook link in every alert	Burn-rate rules and grouping remove duplicate and false pages
24x7 incident leadership	Clear roles and "mitigate first" speed recovery	Faster fixes prevent incidents from spreading into more alerts
Root-cause analysis	Known causes lead to faster future fixes	Permanent fixes stop repeat incidents
Postmortems	Accurate measurement of every time interval	Action items remove sources of future pages
Mentoring 6 engineers	More skilled responders on every shift	Engineers design better alerts and safer releases

6.1 How the Percentages Are Calculated

- **MTTR reduction:** (old average recovery time minus new average recovery time) divided by old average, times 100. Example: (60 minus 33) / 60 = 45%.
- **Paged incident reduction:** (old pages per period minus new pages per period) divided by old pages, times 100. Example: (100 minus 62) / 100 = 38%.
- **Data source:** timestamps and counts recorded in postmortems and the paging system, compared over equal periods before and after the project.

7. Final Recap

In one sentence: we decided what "healthy" means for customers, made the system visible, alerted only when it truly mattered, handled every incident in a calm and structured way at any hour, learned from each one, and trained the team to do the same.

> **Remember the loop**
>
> Measure > Set targets > Watch > Alert early > Respond > Find the root cause > Learn > Improve > Measure again.
>
> Each turn of this loop made the system more reliable, recovery faster (MTTR down 45%) and nights quieter (pages down 38%).